

CVM++

Stack-Based Virtual Machine & Custom Compiler

Abstract

CVM++ is a custom-built, highly optimized, Turing-complete programming language and stack-based Virtual Machine developed entirely from scratch in modern C++17. This project aims to demystify the internal workings of compilers, bytecode generation, and runtime environments. Rather than relying on existing frameworks, CVM++ implements an end-to-end compiler pipeline that translates raw source code into a proprietary 1D bytecode, executing it via a custom-built Virtual CPU.

Contents

1	Introduction and Problem Statement	2
1.1	Project Goals	2
2	System Architecture: The 4-Step Pipeline	2
2.1	Lexical Analysis (The Lexer)	2
2.2	Parsing & AST Generation	2
2.3	Bytecode Compilation	2
2.4	Execution Engine (The Virtual Machine)	3
3	Language Features and Scope	3
3.1	Data Types and Operations	3
3.2	Variable Management	3
3.3	Turing Complete Control Flow	3
3.4	Interactive I/O & Smart Parsing	4
4	Execution Modes & Diagnostic Tooling	4
4.1	Dual Execution Modes	4
4.2	Sample Program	4
4.3	Diagnostic Suite	4
5	Build Instructions and Tech Stack	4
6	References and Acknowledgements	5

1 Introduction and Problem Statement

Most developers utilize high-level languages such as Python, JavaScript, or Java without possessing a deep understanding of how raw text is translated into instructions that a computer can actually execute. The primary objective of this project is to bridge that knowledge gap by building a lightweight, custom scripting language that compiles down to proprietary bytecode, subsequently executed by a custom stack-based Virtual Machine (VM).

1.1 Project Goals

- Design a simple language grammar and an Instruction Set Architecture (ISA).
- Build a Lexer to convert raw source code strings into discrete Tokens.
- Build a Parser to arrange Tokens into an Abstract Syntax Tree (AST).
- Implement a Compiler to flatten the AST into an array of raw Bytecode instructions (Opcodes).
- Build a Virtual Machine (VM) with a stack-based execution loop to execute the Bytecode.

2 System Architecture: The 4-Step Pipeline

CVM++ processes code through a strict, zero-dependency, 4-step pipeline designed for maximum performance and educational clarity.

2.1 Lexical Analysis (The Lexer)

The Lexer processes the raw source code utilizing a suite of character-scanning helper methods. These act as precise, robotic controls (e.g., `advance`, `match`, `peek`) to safely navigate the string, identify keywords, literals, and operators, and efficiently package them into discrete Token objects.

2.2 Parsing & AST Generation

The Parser uses navigational helpers to act as a sliding window, safely stepping through the token stream. It employs a **Recursive Descent** strategy to enforce strict operator precedence and grammar rules, organizing tokens into an Abstract Syntax Tree (AST).

The AST is constructed from "dumb" data container nodes (e.g., `BinaryExpr`) that purely represent structural blueprints. To separate this data from execution logic, operations on the tree are handled via the **Visitor Pattern**, cleanly bridging the gap between parsing and compilation.

2.3 Bytecode Compilation

The Compiler's job is to flatten the 2D AST into a 1D array of bytes (`Chunk`). Operating as a Visitor, it walks the tree node by node:

- **Math (Stack Logic):** Evaluates Left, evaluates Right, then emits the Operator byte.

- **Variables:** Saves names in a constant "vault" and emits bytes to get/set that index.
- **Control Flow:** Employs *backpatching*—emitting a jump with dummy bytes, compiling the branch, then retroactively overwriting the dummy bytes with the true jump distance.

The compilation cleanly terminates by capping the Chunk with an `OP_RETURN` byte before handing it to the VM.

2.4 Execution Engine (The Virtual Machine)

The core VM architecture is built for raw execution speed:

- **The Input:** Takes a direct pointer to the compiled **Chunk**, eliminating memory copying.
- **The Finger (IP):** An Instruction Pointer tracks exactly which byte is executing next.
- **The Sandbox:** A high-performance `std::vector` acts as a temporary stack to instantly push/pop math results.
- **The Vault:** An `std::unordered_map` permanently remembers global variable states.

The Core Mechanism: The VM reads the byte at the IP, utilizes a fast `switch` statement to decode the Opcode, executes the hardwired C++ logic, and advances the IP until execution completes.

3 Language Features and Scope

To maintain simplicity while achieving Turing completeness, the language scope was deliberately focused on fundamental programming paradigms.

3.1 Data Types and Operations

- **Data Types:** Integers and Booleans.
- **Arithmetic & Logic:** Support for simple operations including `+`, `-`, `*`, `/`, `==`, and `<`.

3.2 Variable Management

CVM++ supports dynamic variable management. Global variables can be instantiated and dynamically updated on the fly. The VM securely manages this state in its internal memory.

```
1 let x = 10;
2 x = x + 1;
```

3.3 Turing Complete Control Flow

The language fully supports conditional branching and loops. The compiler handles this by calculating precise bytecode jump offsets (`OP_JUMP`, `OP_JUMP_IF_FALSE`, `OP_LOOP`) to route execution instantly.

- `if/else` statements.
- `while` loops.

3.4 Interactive I/O & Smart Parsing

Output is handled via the `print` keyword. Input is handled dynamically using the `input` keyword, which pauses execution to capture user data. The VM employs "Smart Parsing" to automatically evaluate inputs as integers for arithmetic, securely falling back to strings where appropriate.

4 Execution Modes & Diagnostic Tooling

4.1 Dual Execution Modes

CVM++ can be executed in two primary environments:

1. **File Mode (Script Runner):** Run entire `.cvm` scripts directly from the filesystem (e.g., `./cvm scripts/example.cvm`).
2. **Interactive REPL (Read-Eval-Print Loop):** A line-by-line interactive prompt for rapid prototyping and testing.

4.2 Sample Program

The following script demonstrates variable assignment, looping, I/O, and arithmetic evaluation within CVM++:

```
1 let count = 0;
2 while (count < 3) {
3     let add = input;
4     count = count + add;
5     print count;
6 }
```

4.3 Diagnostic Suite

To provide deeper insights into the compilation process, several targeted modular executables were built:

- `cvm_lexer`: Visualizes the raw Token stream.
- `cvm_parser`: Pretty-prints the 2D Abstract Syntax Tree structure.
- `cvm_compiler`: Disassembles the bytecode and prints the Constant Pool.
- `cvm_vm`: Traces the live CPU stack during bytecode execution.
- `cvm_final`: Runs the complete pipeline silently without debug logs for maximum execution speed.

5 Build Instructions and Tech Stack

The project utilizes **CMake** for robust cross-platform building. The core logic is written entirely in **C++17**.

To enforce maximum execution speed and rigorous code quality, the build configuration explicitly utilizes the `-O3`, `-Wall`, `-Wextra`, and `-pedantic` flags.

```
1 # Configure CMake
2 cmake -B build -S .
3
4 # Compile the highly optimized executables
5 cmake --build build
```

6 References and Acknowledgements

The architectural foundation and theoretical backing of this project were heavily inspired by the definitive guide on the subject:

- Robert Nystrom: *Crafting Interpreters*. <https://craftinginterpreters.com/>

This resource was instrumental in providing the foundational concepts necessary to engineer the Lexer, Parser, Bytecode Compiler, and the underlying Virtual Machine.